

PySpark Bucketing and Segmentation

Core Concept: Bucketing divides continuous values into categories (Gold, Silver, Bronze) to simplify analysis and support business decisions.

Methods Covered:

1. Conditional Logic (when/otherwise)
2. SQL CASE Statement
3. Bucketizer (MLlib)
4. Quantile-based Segmentation
5. Window-based Ranking

1. Load Customer and Order Data

```
In [ ]: from pyspark.sql.functions import col, sum as _sum, when, count, round
        as _round

        # Load customer and order data
        customers = spark.read.format("csv").option("header",
            "true").load("/Volumes/workspace/default/customers")
        orders = spark.read.format("csv").option("header",
            "true").load("/Volumes/workspace/default/customers/orders.csv")

        # Cast total_amount to double for calculations
        orders = orders.withColumn("total_amount",
            col("total_amount").cast("double"))

        # Calculate total spend per customer
        customer_spend = orders.groupBy("customer_id").agg(
            _sum("total_amount").alias("total_spend"),
            count("order_id").alias("order_count")
        )

        # Join with customer details
        customer_data = customers.join(customer_spend, "customer_id", "left")

        # Fill nulls for customers with no orders
        customer_data = customer_data.fillna({"total_spend": 0, "order_count":
            0})

        print(f"Total customers: {customer_data.count()}")
        display(customer_data.select("customer_id", "first_name", "last_name",
            "city", "total_spend",
            "order_count").orderBy(col("total_spend").desc()).limit(10))
```

Total customers: 160

customer_id	first_name	last_name	city	total_spend	order_count
8	4	2024-01-22	null	169.94	2
8	James	Martinez	Des Moines	169.94	2
8	5	8	null	169.94	2
3	2	2	null	169.94	2
3	Olivia	Brown	Greenville	169.94	2
3	2	2024-01-17	null	169.94	2

Databricks visualization. Run in Databricks to view.

Databricks visualization. Run in Databricks to view.

2. Method 1: Conditional Logic (when/otherwise)

Most Common Method - Uses PySpark's `when()` function to apply business rules.

```
In [ ]: # Method 1: Conditional Logic using when/otherwise
df_method1 = customer_data.withColumn(
    "segment",
    when(col("total_spend") > 100, "Gold")
    .when((col("total_spend") >= 50) & (col("total_spend") <= 100),
    "Silver")
    .otherwise("Bronze")
)

print("Method 1: Conditional Logic Segmentation")
display(df_method1.select("customer_id", "first_name", "last_name",
"total_spend", "segment").orderBy(col("total_spend").desc()))

# Group by segment and count
segment_counts1 = df_method1.groupBy("segment").agg(
    count("customer_id").alias("customer_count"),
    _round(_sum("total_spend"), 2).alias("total_revenue"),
    _round(_sum("total_spend") / count("customer_id"),
    2).alias("avg_spend_per_customer")
).orderBy(col("total_revenue").desc())

print("\nSegment Summary:")
display(segment_counts1)
```

Method 1: Conditional Logic Segmentation

customer_id	first_name	last_name	total_spend	segment
8	5	8	169.94	Gold
3	Olivia	Brown	169.94	Gold
3	2	2	169.94	Gold
8	4	2024-01-22	169.94	Gold
3	2	2024-01-17	169.94	Gold
8	James	Martinez	169.94	Gold
10	Lucas	Rodriguez	150.95	Gold
5	Noah	Williams	150.95	Gold

Segment Summary:

segment	customer_count	total_revenue	avg_spend_per_customer
Gold	42	5386.08	128.24
Silver	36	2793.96	77.61
Bronze	82	2405.4	29.33

```
In [ ]: # Visualize Method 1 segment summary
print("Method 1: Segment Summary Visualizations")
display(segment_counts1)
```

Method 1: Segment Summary Visualizations

segment	customer_count	total_revenue	avg_spend_per_customer
Gold	42	5386.08	128.24
Silver	36	2793.96	77.61
Bronze	82	2405.4	29.33

Databricks visualization. Run in Databricks to view.

Databricks visualization. Run in Databricks to view.

```
In [ ]: # Visualize Method 2 segment summary
print("Method 2: Segment Summary Visualizations")
display(segment_counts2)
```

Method 2: Segment Summary Visualizations

segment	customer_count	total_revenue
Gold	42	5386.08
Silver	36	2793.96
Bronze	82	2405.4

Databricks visualization. Run in Databricks to view.

Databricks visualization. Run in Databricks to view.

```
In [ ]: # Visualize Method 3 segment summary
print("Method 3: Segment Summary Visualizations")
display(segment_counts3)
```

Method 3: Segment Summary Visualizations

segment	customer_count	total_revenue
Gold	42	5386.08
Silver	36	2793.96
Bronze	82	2405.4

Databricks visualization. Run in Databricks to view.

Databricks visualization. Run in Databricks to view.

```
In [ ]: # Visualize Method 4 segment summary
print("Method 4: Segment Summary Visualizations")
display(segment_counts4)
```

Method 4: Segment Summary Visualizations

segment	customer_count	total_revenue
Gold	60	6959.22
Silver	54	2780.34
Bronze	46	845.88

Databricks visualization. Run in Databricks to view.

Databricks visualization. Run in Databricks to view.

```
In [ ]: # Visualize Method 5 segment summary
print("Method 5: Segment Summary Visualizations")
display(segment_counts5)
```

Method 5: Segment Summary Visualizations

```
/databricks/python/lib/python3.12/site-
packages/pyspark/sql/connect/expressions.py:1160: UserWarning:
WARN WindowExpression: No Partition Defined for Window
operation! Moving all data to a single partition, this can
cause serious performance degradation.
warnings.warn(
```

segment	customer_count	total_revenue
Gold	54	6465.72
Silver	48	2794.08
Bronze	58	1325.64

Databricks visualization. Run in Databricks to view.

Databricks visualization. Run in Databricks to view.

3. Method 2: SQL CASE Statement

SQL-Based Approach - Uses SQL CASE expression for segmentation.

```
In [ ]: # Method 2: SQL CASE Statement
customer_data.createOrReplaceTempView("customers_temp")

df_method2 = spark.sql("""
    SELECT
        *,
        CASE
            WHEN total_spend > 100 THEN 'Gold'
            WHEN total_spend BETWEEN 50 AND 100 THEN 'Silver'
            ELSE 'Bronze'
        END AS segment
    FROM customers_temp
""")

print("Method 2: SQL CASE Segmentation")
display(df_method2.select("customer_id", "first_name", "last_name",
    "total_spend", "segment").orderBy(col("total_spend").desc()))

# Group by segment
segment_counts2 = df_method2.groupBy("segment").agg(
    count("customer_id").alias("customer_count"),
    _round(_sum("total_spend"), 2).alias("total_revenue")
).orderBy(col("total_revenue").desc())

print("\nSegment Summary:")
display(segment_counts2)
```

Method 2: SQL CASE Segmentation

customer_id	first_name	last_name	total_spend	segment
8	5	8	169.94	Gold
3	Olivia	Brown	169.94	Gold
3	2	2	169.94	Gold
8	4	2024-01-22	169.94	Gold
3	2	2024-01-17	169.94	Gold
8	James	Martinez	169.94	Gold
10	Lucas	Rodriguez	150.95	Gold
5	Noah	Williams	150.95	Gold

Segment Summary:

segment	customer_count	total_revenue
Gold	42	5386.08
Silver	36	2793.96

segment	customer_count	total_revenue
Bronze	82	2405.4

4. Method 3: Bucketizer (MLlib)

Machine Learning Approach - Uses MLlib's Bucketizer for creating bins with defined splits.

```

In [ ]: from pyspark.ml.feature import Bucketizer

# Method 3: Bucketizer from MLlib
# Define splits: [-inf, 50, 100, +inf]
splits = [-float("inf"), 50, 100, float("inf")]

bucketizer = Bucketizer(splits=splits, inputCol="total_spend",
outputCol="bucket_index")
df_method3 = bucketizer.transform(customer_data)

# Map bucket index to segment names
df_method3 = df_method3.withColumn(
    "segment",
    when(col("bucket_index") == 0, "Bronze")
    .when(col("bucket_index") == 1, "Silver")
    .when(col("bucket_index") == 2, "Gold")
    .otherwise("Unknown")
)

print("Method 3: Bucketizer Segmentation")
display(df_method3.select("customer_id", "first_name", "last_name",
"total_spend", "bucket_index",
"segment").orderBy(col("total_spend").desc()))

# Group by segment
segment_counts3 = df_method3.groupBy("segment").agg(
    count("customer_id").alias("customer_count"),
    _round(_sum("total_spend"), 2).alias("total_revenue")
).orderBy(col("total_revenue").desc())

print("\nSegment Summary:")
display(segment_counts3)

```

Method 3: Bucketizer Segmentation

customer_id	first_name	last_name	total_spend	bucket_index
3	Olivia	Brown	169.94	2.0
3	2	2	169.94	2.0
3	2	2024-01-17	169.94	2.0
8	5	8	169.94	2.0
8	James	Martinez	169.94	2.0
8	4	2024-01-22	169.94	2.0

Segment Summary:

segment	customer_count	total_revenue
Gold	42	5386.08
Silver	36	2793.96
Bronze	82	2405.4

5. Method 4: Quantile-based Segmentation

Data-Driven Approach - Splits are determined by data distribution (33rd and 66th percentiles).

```
In [ ]: # Method 4: Quantile-based Segmentation
# Calculate 33rd and 66th percentiles
quantiles = customer_data.approxQuantile("total_spend", [0.33, 0.66],
0.01)
print(f"Quantile thresholds: 33rd percentile = {quantiles[0]:.2f}, 66th
percentile = {quantiles[1]:.2f}")

# Apply quantile-based segmentation
df_method4 = customer_data.withColumn(
    "segment",
    when(col("total_spend") >= quantiles[1], "Gold")
    .when((col("total_spend") >= quantiles[0]) & (col("total_spend") <
quantiles[1]), "Silver")
    .otherwise("Bronze")
)

print("\nMethod 4: Quantile-based Segmentation")
display(df_method4.select("customer_id", "first_name", "last_name",
"total_spend", "segment").orderBy(col("total_spend").desc()))

# Group by segment
segment_counts4 = df_method4.groupBy("segment").agg(
    count("customer_id").alias("customer_count"),
    _round(_sum("total_spend"), 2).alias("total_revenue")
).orderBy(col("total_revenue").desc())

print("\nSegment Summary:")
display(segment_counts4)
```

Quantile thresholds: 33rd percentile = 39.98, 66th percentile = 82.25

Method 4: Quantile-based Segmentation

customer_id	first_name	last_name	total_spend	segment
8	5	8	169.94	Gold
3	Olivia	Brown	169.94	Gold
3	2	2	169.94	Gold
8	4	2024-01-22	169.94	Gold
3	2	2024-01-17	169.94	Gold
8	James	Martinez	169.94	Gold
10	Lucas	Rodriguez	150.95	Gold
5	Noah	Williams	150.95	Gold

Segment Summary:

segment	customer_count	total_revenue
Gold	60	6959.22
Silver	54	2780.34
Bronze	46	845.88

6. Method 5: Window-based Ranking

Percentile Rank Approach - Uses percent_rank window function to assign relative positions.

```
In [ ]: from pyspark.sql.window import Window
from pyspark.sql.functions import percent_rank

# Method 5: Window-based Ranking
window = Window.orderBy("total_spend")
df_method5 = customer_data.withColumn("rank_pct",
percent_rank().over(window))

# Segment based on percentile rank
df_method5 = df_method5.withColumn(
    "segment",
    when(col("rank_pct") >= 0.66, "Gold")
    .when((col("rank_pct") >= 0.33) & (col("rank_pct") < 0.66),
"Silver")
    .otherwise("Bronze")
)

print("Method 5: Window-based Ranking Segmentation")
display(df_method5.select("customer_id", "first_name", "last_name",
"total_spend", "rank_pct",
"segment").orderBy(col("total_spend").desc()))

# Group by segment
segment_counts5 = df_method5.groupBy("segment").agg(
    count("customer_id").alias("customer_count"),
    _round(_sum("total_spend"), 2).alias("total_revenue")
).orderBy(col("total_revenue").desc())

print("\nSegment Summary:")
display(segment_counts5)
```

Method 5: Window-based Ranking Segmentation

```
/databricks/python/lib/python3.12/site-
packages/pyspark/sql/connect/expressions.py:1160: UserWarning:
WARN WindowExpression: No Partition Defined for Window
operation! Moving all data to a single partition, this can
cause serious performance degradation.
    warnings.warn(
```

customer_id	first_name	last_name	total_spend	rank_pct
3	Olivia	Brown	169.94	0.9685534591194
8	James	Martinez	169.94	0.9685534591194
3	2	2024-01-17	169.94	0.9685534591194
8	4	2024-01-22	169.94	0.9685534591194
3	2	2	169.94	0.9685534591194
8	5	8	169.94	0.9685534591194



Segment Summary:

segment	customer_count	total_revenue
Gold	54	6465.72
Silver	48	2794.08
Bronze	58	1325.64

7. Comparison of All Methods

Compare how different methods distribute customers across segments.

Practice Tasks

1.  Create Gold/Silver/Bronze segmentation using conditional logic
2.  Group data by segment and count customers
3.  Try quantile-based segmentation
4.  Compare results of different methods
5.  Reflect: which method is most useful and why?

```

In [ ]: from pyspark.sql.functions import lit

# Collect all segment summaries and add method labels
# Ensure all DataFrames have the same columns in the same order
comparison = (
    segment_counts1.withColumn("method", lit("1. Conditional
Logic")).select("method", "segment", "customer_count", "total_revenue",
"avg_spend_per_customer")
    .union(segment_counts2.withColumn("avg_spend_per_customer",
_round(col("total_revenue") / col("customer_count"),
2)).withColumn("method", lit("2. SQL CASE")).select("method", "segment",
"customer_count", "total_revenue", "avg_spend_per_customer"))
    .union(segment_counts3.withColumn("avg_spend_per_customer",
_round(col("total_revenue") / col("customer_count"),
2)).withColumn("method", lit("3. Bucketizer")).select("method",
"segment", "customer_count", "total_revenue", "avg_spend_per_customer"))
    .union(segment_counts4.withColumn("avg_spend_per_customer",
_round(col("total_revenue") / col("customer_count"),
2)).withColumn("method", lit("4. Quantile-based")).select("method",
"segment", "customer_count", "total_revenue", "avg_spend_per_customer"))
    .union(segment_counts5.withColumn("avg_spend_per_customer",
_round(col("total_revenue") / col("customer_count"),
2)).withColumn("method", lit("5. Window Ranking")).select("method",
"segment", "customer_count", "total_revenue", "avg_spend_per_customer"))
)

print("Comparison of All Segmentation Methods:")
display(comparison.select("method", "segment", "customer_count",
"total_revenue").orderBy("method", col("total_revenue").desc()))

# Pivot to see side-by-side comparison
print("\nSide-by-Side Customer Count Comparison:")
pivot_comparison =
comparison.groupBy("method").pivot("segment").agg(_sum("customer_count")
)
display(pivot_comparison.select("method", "Gold", "Silver",
"Bronze").orderBy("method"))

```

Comparison of All Segmentation Methods:

```

/databricks/python/lib/python3.12/site-
packages/pyspark/sql/connect/expressions.py:1160: UserWarning:
WARN WindowExpression: No Partition Defined for Window
operation! Moving all data to a single partition, this can
cause serious performance degradation.
warnings.warn(

```

method	segment	customer_count	total_revenue
1. Conditional Logic	Gold	42	5386.08
1. Conditional Logic	Silver	36	2793.96
1. Conditional Logic	Bronze	82	2405.4

method	segment	customer_count	total_revenue
2. SQL CASE	Gold	42	5386.08
2. SQL CASE	Silver	36	2793.96
2. SQL CASE	Bronze	82	2405.4



Databricks visualization. Run in Databricks to view.

Databricks visualization. Run in Databricks to view.

Side-by-Side Customer Count Comparison:

method	Gold	Silver	Bronze
1. Conditional Logic	42	36	82
2. SQL CASE	42	36	82
3. Bucketizer	42	36	82
4. Quantile-based	60	54	46
5. Window Ranking	54	48	58

8. Reflection Questions

Why do we convert continuous values into categories?

- * Simplifies analysis and decision-making
- * Makes data more interpretable for business stakeholders
- * Enables targeted marketing and personalized strategies
- * Reduces computational complexity in some scenarios

What is the difference between business segmentation and technical bucketing?

- * **Business segmentation:** Based on domain knowledge and business rules (e.g., Gold > \$100)
- * **Technical bucketing:** Based on data distribution (quantiles, percentiles, statistical methods)

When would fixed thresholds fail?

- * When data distribution changes over time (inflation, market shifts)
- * When applying the same thresholds across different markets/regions
- * When outliers skew the distribution significantly
- * When the business context changes (e.g., economy shifts)

How does quantile-based segmentation differ from fixed rules?

- * **Fixed rules:** Same thresholds regardless of data (e.g., always \$50 and \$100)
- * **Quantile-based:** Adapts to data distribution (e.g., top 33% are always Gold)
- * Quantile ensures balanced segment sizes
- * Fixed rules ensure consistent business meaning

Which method would you use in real-world projects?

It depends on the use case:**1. Conditional Logic / SQL CASE** (Methods 1 & 2)

- * **When:** You have clear business rules and thresholds
- * **Example:** "Premium customers spend over \$10,000"
- * **Pros:** Easy to understand, consistent meaning
- * **Cons:** May create imbalanced segments

2. Bucketizer (Method 3)

- * **When:** Integrating with ML pipelines
- * **Example:** Feature engineering for model training
- * **Pros:** Works seamlessly with MLlib
- * **Cons:** Requires defining splits upfront

3. Quantile-based (Method 4)

- * **When:** You want balanced segment sizes
- * **Example:** "Top 20% of customers"
- * **Pros:** Adapts to data distribution
- * **Cons:** Thresholds change as data changes

4. Window Ranking (Method 5)

- * **When:** You need relative positioning
- * **Example:** Leaderboards, percentile-based rewards
- * **Pros:** Precise percentile control
- * **Cons:** More computationally expensive

Recommendation: Start with **Conditional Logic** (Method 1) for most business use cases. Use **Quantile-based** (Method 4) when you need balanced segments or when thresholds are unclear.

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).